Chapter 10. Incident Management and On-Call

By Dan Dennison, David Huska, Garrett Plasky, Jennifer Mace, and Michelle Brush with Garrett Holthaus

Complex systems fail. This is an unavoidable reality of software engineering at scale. When they do, organized incident management and on-call practices are the difference between a system that is quickly and smoothly restored to full operation and one that remains unhealthy or unavailable for longer than necessary.

In this chapter, we provide a practical guide to navigating broken systems. It covers foundational concepts around how to set up sustainable on-call rotations and organize your incident response to effectively triage, mitigate, and resolve outages. We'll discuss our model for incident response at Google, identify common organizational pitfalls, and offer technical and cultural solutions to them.

We'll also specifically address the human element: the specific, learnable skills required for incident response and how to ensure your team is able to respond effectively when everything seems to be going wrong.

Finally, we'll examine how AI and ML fit into incident response, exploring their current and expected capabilities and risks in this domain. The chapter wraps up with a case study illustrating how Etsy applies the principles discussed in this chapter to its incident management practices.

Incidents, Outages, and Bugs

Let's start by defining the term *incident* and how it relates to outages and software bugs. An incident is any event that suggests one of the following:

- Users are having a bad experience.
- Important system properties are degraded.
- An outage is occurring or imminent.

The term “outage” is typically reserved for when a service or critical feature is unavailable or so severely degraded that it cannot fulfill its purpose. In contrast, software bugs are unexpected system behaviors that may or may not reach the level of breaking the service. Not all bugs are incidents, and not all incidents are outages. Yet, the distinction between these categories is often unclear when an issue is first detected.

To handle the ambiguity, it’s good practice to establish clear criteria for distinguishing bugs, incidents, and outages. As we’ll see, responding to an incident is not without the risk of compounding the situation by introducing new problems. Teams should therefore set guidelines for how people are expected to behave in each case. The risk and urgency of action should reflect the threat to the organization. For example, an outage might call for immediate and active engagement, whereas a bug might be handled by filing a tracking ticket.

In large organizations, politics can influence the classification of an incident. If the organization is counting outages as a performance metric, there could be pressure not to label an incident an outage. Alternatively, if calling something an outage allows teams to bypass a policy or restriction, they may be incentivized to overescalate. Having clear criteria in place can mitigate these dynamics.

To avoid politics playing a prominent role in incidents, a core aspect of Google’s approach is that anyone can declare an incident and escalate it to an outage. To ensure there are no unintended consequences for premature escalation, we can label an incident as a false positive if we later discover the situation wasn’t as bad as we initially thought.

Foundations of On-Call

Incident management encompasses the activities required to mitigate and ultimately resolve an incident. While not all teams require an active on-call rotation to handle incidents, the incident management process is most effective when performed by an on-call team.

On-call is effectively a mechanism for coordinating urgent work. Determining what work falls under the responsibility of an on-call rotation varies, but it often includes responding to automated alerts, handling urgent operational tasks, and managing incidents. In some cases, an on-call rotation may handle customer support cases or fix bugs. One that primarily responds to routine customer service requests (account questions, quota increases, etc.) is unlikely to manage incidents.

Some organizations forego incident management rotations and rely instead on best effort structures, such as notifying the whole team or a team leader when something is wrong. The problem with notifying the whole team is that it can lead to the bystander effect: no one responds because everyone assumes someone else will. Conversely, relying on a single individual, such as a manager or tech lead, makes that person a single point of failure. This is why structured response mechanisms like the ones described throughout this chapter are preferred. They ensure that there is always someone with the right skills and training who knows they are expected to respond in a timely manner, and that these skills are applied effectively before, during, and after an incident.

We'll start by identifying key factors to consider when structuring an on-call rotation. "Incident Response Teams" discusses another approach for structuring incident response.

Structuring an On-Call Rotation

Designing an on-call rotation for a service involves a number of considerations, such as paging load, hours of availability, response time requirements, system properties, and customer needs. Each of these factors influences the ultimate shape and size of the on-call team.

Optimal Paging Load

The primary concern for on-call is the anticipated load. Ideally, an SRE should handle at most two incidents per shift, assuming each incident lasts four to six hours each. Beyond this, the responders become overloaded and won't have time to investigate the incident thoroughly.

A heavier load commonly leads to a degraded response. When engineers are constantly interrupted or overwhelmed, remediations are slower, investigations are shallower, and the time spent learning from the incident is shorter. High pager volume increases the risk of critical alerts being missed or responses delayed as engineers experience "alert fatigue."

Conversely, too few incidents can create underload, which can also slow and degrade incident response due to lack of experience and familiarity with the system. Smooth and effective incident response requires practice—knowing exactly which tools to use, how to use them, and when. A service that rarely pages might not justify SRE support at all, as the development team will be much better prepared to respond because they frequently

interact with the codebase. That said, it's more common for teams to find themselves dealing with too many incidents rather than too few.

Hours of Availability

The next concern for designing an on-call rotation is the hours of the day people are on-call. Twenty-four-hour coverage is not essential for every service or user base, and it's expensive in terms of both the cost to the organization and the toll on the responders—especially if the rotation has a small number of participants or doesn't span many time zones. On-call coverage should be justified by business risk. Here are some typical on-call schedules:

Business hours only (8 a.m. to 5 p.m.)

This is generally not considered on-call work but rather a rotation of on-duty engineers to respond quickly to issues.

Daytime hours only (6 a.m. to 8 p.m.)

This schedule may be a good choice for those services that support a user base in adjacent time zones, or where problems detected overnight do not require staffing multiple sites or waking up an on-caller to resolve them.

Daylight hours only (e.g., 5:30 a.m. CET to 8:30 p.m. ET)

This schedule provides nearly 21 hours of coverage without anyone having to work during late-night hours, but it requires teams staffed in multiple locations. The exact coverage depends on the time zones involved.

Full 24-hour schedule

This schedule is essential for services requiring an immediate, around-the-clock response. To manage it sustainably without exceeding maximum operational load limits, organizations might employ a "follow-the-sun" approach using geographically diverse, appropriately sized teams (see Chapter 6). Alternatively, they may provide on-call engineers with flexible working hours, compensatory time, or other measures to support long-term team health.

Note that local jurisdictions sometimes have labor regulations regarding work outside of typical business hours, which may add other scheduling considerations.

Response Time to Notifications

Different services have different requirements for on-call response. Some require the engineer to join a call or have hands on the keyboard within five minutes of an alert. Others are comfortable with up to 30-minute or even 60-minute response times. The response time selected depends on a number of factors, such as financial and reputational damage to the organization when the service is down, the service's SLO, regulatory compliance, and so on. The availability of automated remedial options in production may also shape response time.

In practice, any rotation that requires a response in three minutes or less needs to have at least two engineers ready and able to respond within that time frame in order to ensure coverage for basic human needs. Five-minute response schedules are a bit more forgiving. However, both require handoff during commutes and explicit planning for periods of personal unavailability. In this situation, a page might go simultaneously to all on-call engineers, although a common configuration is to designate an alerting order: alerts go first to the primary engineer, then the secondary if the alert is not acknowledged, and so on. These response windows also require the on-caller to carry a phone, a laptop, or both at all times during their shift, depending on whether they are required to begin actively working on the incident or just to acknowledge it within the required response time.

If the response requirement is 30 minutes or more, on-callers have the flexibility to finish a commute, run errands, and even go out to dinner with friends or family while on-call. Depending on the expectations, they might not even need to carry a laptop with them.

System Properties

Some services can tolerate lower response times and/or less coverage because the system is designed in a way that service degradation or even downtime can be temporarily tolerated. This may be the case if the service is not in the serving path of any critical user interactions, dependent systems are able to gracefully degrade if the service fails, or the service has self-healing properties. For example, a batch service that runs once per day to clean up data may only need someone to respond within business hours. As another example, a recommendation engine may not be critical to the system if the system can leverage intelligent defaults when the engine is down.

Customer Requirements

Customer requirements may add constraints to the design of an on-call rotation. For example, some customers may request that responders be physically located within their region or country. Other customers may request that the engineers meet citizenship, immigration, or employment status requirements or have specific certifications or clearances.

Organizations that agree to such requests will have to build response teams that satisfy the customer's location, time zone, and personnel requirements. However, with more geographically diverse teams, additional constraints often affect on-call schedules, such as differential pay for on-call work, limits on consecutive days on-call, limits on hours worked per week, or restricted times in which on-call is not permitted.

Satisfying a wide variety of requirements may require creativity, including relying on other teams, functions, or even companies to achieve the necessary geographical diversity. With this approach, the main challenge is having sufficient tooling, documentation, policies, and training. Additionally, more complex incidents will likely still require occasional escalation to subject matter experts. The escalations themselves may have further restrictions.

Additional Implementation Considerations

Planning sustainable on-call rotations requires balancing numerous considerations, which becomes labor-intensive when performed manually. A constraint solver can ensure compliance with local laws and policies without overloading planners. Google utilizes an internal on-call optimizer to manage these variables and ensure fairness. Similar tools are available publicly to assist with this task.

If no solution exists given the constraints, the organization will have to decide what constraints to relax. For example, if it's infeasible to staff a dual-site, follow-the-sun model for 24×7 response, the organization can compromise on response time requirements, offer compensatory time, or have a nontraditional work schedule. While these compromises are not ideal, they are better than risking employee burnout by expecting constant, around-the-clock availability for extended durations.

Escalation Rotations

In addition to the core on-call rotations for services, organizations often need to create escalation rotations. The most common is an incident response team (see "Incident

Response Teams") staffed with seasoned responders who are paged into more complex and longer-running incidents that span teams and even functions. Because escalation rotations are invoked less frequently than product rotations, they can be scheduled more flexibly. However, the cognitive load of these escalations is high. Sustainability is an important concern for these rotations as well.

INCIDENT MANAGEMENT AT GOOGLE

Incident Management at Google (IMAG) is Google's standardized framework for effectively dealing with all types of incidents, including service outages. Adapted from the Incident Command System developed by emergency response agencies in the United States and widely used internationally, it establishes an operational structure with clear roles, responsibilities, and communication channels that expand or shrink easily to meet the needs of the immediate situation.

There are three primary roles:

Incident commander

Leads and coordinates the overall response, assigns tasks, and makes decisions

Communications lead

Ensures communication channels are established, documents the state of the outage, and regularly updates stakeholders

Operations lead

Performs the investigations and actions required to triage, mitigate, and resolve the outage at the direction of the incident commander

The incident commander owns all roles until they are delegated. For simple outages, it is possible for the incident commander to perform all roles if they are sure that they can communicate effectively, stay in control of the incident response, and work with other responders to resolve the incident. If they feel that they cannot perform all of these roles simultaneously, they need to activate the other responder roles.

In more complex incidents, there is only one incident commander for the overall incident, but depending on the scale, complexity, and other circumstances, multiple people might serve in the other roles. For example, if an incident is impacting customers, there can be both an internal communications lead and an external communications lead. The information presented to these audiences is obviously different.

Google tracks incidents on an internal incident management platform, where anyone responding to the incident can see and update status, while the system ensures established practices are maintained. The platform follows the same detect, triage, mitigate, resolve, and learn approach outlined in "The Lifecycle of an Incident", and allows transition to any state at any time because of the often nonlinear flow of incidents.

For complex incidents, Google sometimes creates a tracking hierarchy. At the top is a parent incident, with the overall incident commander coordinating the full response—typically a member of an incident response team. Underneath that is a tree of child incidents, with each service having its own responders. This allows a "divide and conquer" approach while still making sure the response is fully coordinated by a single person, avoiding confusion about who is in charge and letting the people most familiar with their services make the decisions for their areas of expertise.

Google's approach to incident management follows a key principle: move the decision to the information. We recognize that in high-stress, high-urgency situations, the people with the most context about the system should be making the calls. The incident commander of an outage is fully empowered to make decisions, take actions, and bring in any person they need to resolve the situation.

Incident Response Teams

An incident response team (IRT) is typically a group of experienced engineers, often seasoned SREs, who provide support and coordination during significant, complex, or wide-reaching incidents. Their primary goal is to efficiently resolve major issues by quickly mitigating the impact and ensuring effective communication among all involved stakeholders.

If SREs are good at handling outages, why are IRTs even necessary? They usually get involved through escalation when an incident spans multiple services or teams, or exceeds the capacity of the initial responders. IRTs may even be paged directly if the incident matches certain criteria. These teams provide an important complement to general SRE response: they act as a higher-level coordinator of more specialized SRE teams.

IRTs at Google often differ structurally and/or operationally depending on their area of responsibility, but most share these characteristics:

Incident command

: Often taking on the incident commander role to manage the overall response

Coordination

: Orchestrating the efforts of multiple teams and responders, including triage

Communication

Ensuring clear and timely communication with other teams and to stakeholders, including up the leadership chain and often to external customers

Escalation

Knowing who to page and how to escalate further when needed, potentially involving other IRTs or subject matter experts

Resource allocation

Finding and allocating resources to address the incident

Technical expertise

Providing broad technical knowledge to guide troubleshooting and mitigation efforts

Organization

The optimal structure for an incident response team depends heavily on the scale of the organization and the complexity of its systems. While Google employs a variety of IRT models tailored to different product areas and functions (see "SRE Managing Real-World Safety: An Example with Waymo"), external organizations should choose a style that matches their operational situation.

Large organizations: Federating incident response

Large organizations with broad product portfolios and complex infrastructure often find that a single, centralized IRT is insufficient. The coordination overhead and the breadth of domain knowledge required become too great. The NIST Computer Security Incident Handling Guide defines both centralized and distributed incident response team structures;[1] however, in practice, large technology companies often combine these into a hybrid or federated model to balance central coordination with localized expertise.

In this federated model, the organization maintains a "global" or "core" IRT alongside multiple IRTs specific to particular product areas (PAs):

Global IRT

This team handles incidents that affect core infrastructure or span multiple business units, functioning as a central coordinating body. Its members are experts in large-scale coordination and crisis management. At Google, Tech IRT fulfills this role, responding to major outages that impact fundamental infrastructure.

PA-specific IRTs

These teams are embedded within specific product areas or business units. They possess deeper domain context than the core IRT but broader systems awareness than service teams. They handle large incidents within their specific domains. At Google, teams like Cloud IRT, Search IRT, and YouTube SRE IRT exemplify this pattern, providing specialized response for their respective platforms.

Several best practices contribute to successfully introducing incident response teams to an established organization:

Define clear escalation triggers

Avoid ambiguity about when to engage which team. Use objective criteria such as the number of services impacted, duration of impact, or customer severity.

Maintain a unified command framework

Even with multiple teams, the roles (incident commander, communications lead, operations lead) must remain consistent. A responder moving from a service team to a PA-specific IRT should encounter the same operational language and expectations.

Foster a community of practice

Regular training and joint exercises between the core IRT and PA-specific IRTs help build trust and ensure smooth handoffs during real incidents.

Beware of coordination overhead

Adding more teams can increase communication latency. Ensure that the engagement of an IRT clarifies command rather than adding another layer of bureaucracy.

Prevent skill atrophy

Service teams must not become overreliant on IRTs. The IRT's primary value is coordinating the response and handling complex failure modes, not basic firefighting. Service teams must remain capable of handling routine incidents.

Small organizations: Bootstrapping the first IRT

For smaller organizations, dedicated, full-time IRT staffing is rarely justifiable. The pager load is typically too low to maintain skills, and the cost is prohibitive. Instead, the goal is

to create a repeatable pattern for handling "black swan" events without dedicated headcount. An effective pattern for a smaller organization is a per-incident swarm model.

This relies on a pool of experienced senior engineers who volunteer or are nominated to be on an escalation rotation. They perform their normal duties but are paged into major incidents to provide coordination and high-stakes decision-making support. This model leverages existing expertise without the cost of a dedicated team.

Here are some best practices for introducing an IRT in a smaller organization:

Empower through inversion of authority

The single most important factor in bootstrapping an IRT is cultural, not structural. The organization must explicitly empower the incident responder (acting as incident commander) to make decisions and direct resources regardless of the normal management hierarchy.

Provide business context

Ensure that responders have access to business context (e.g., which customers are critical, what data is sensitive) before the incident occurs. This avoids the antipattern of relying on executives to make technical decisions in the moment because they are the only people with the necessary context.

Start small

Begin with a small pool of trusted, senior engineers. Focus on establishing the process and building confidence before expanding the pool.

Avoid "incident theater"

Do not equate a large number of people on an incident bridge with effective management. The response should be as large as it has to be and as small as it can be to be effective.

Protect volunteers from burnout

Because participants have day jobs, ensure that their IRT participation is recognized in performance reviews and that they are not penalized for focusing on an incident instead of their normal project work.

Influence on Organizational Technical Policy

Beyond managing crises, an IRT is uniquely positioned to influence broad technical policy. Because these teams see the most complex failures across the entire system, they can spot systemic risks that individual service teams might miss. They become powerful advocates for architectural improvements, standard platforms, and better observability practices across the company.

Potential Drawbacks

Implementing IRTs involves trade-offs because it can introduce some risks. Used appropriately, the benefits outweigh them; however, here are some risks to be aware of:

Coordination overhead

Introducing an IRT adds more people and a layer of coordination. This requires clear roles and communication to avoid confusion. Google minimizes this overhead by requiring IRTs to use the same framework used by incident response teams across the company (see the sidebar "Incident Management at Google").

Handoffs

Transferring context from the initial responders to the IRT (and possibly back again once the severity of the incident is reduced) must be efficient and effective. The IRT is typically not the owner of the affected service and may require more context than a responder who regularly handles outages on the service.

Engagement latency

Teams might delay paging the IRT, hoping to solve the problem themselves but potentially prolonging and worsening the impact. The value of IRTs is maximized when they are engaged early.

Availability

Some IRTs, especially smaller or best-effort ones, rely on the availability of key experienced individuals. If these people are unavailable when needed, the purpose of the IRT is defeated.

Skill atrophy

If the IRT becomes a default escalation for any moderately difficult issue, service teams may lose their edge in handling incidents independently. An IRT should be

viewed as a partner and a resource for exceptional circumstances, not a replacement for existing service-level on-call capabilities.

Political considerations

Friction can arise if service teams or even whole product divisions feel the IRT is infringing on their autonomy. This is especially true when IRTs are required to make difficult triage decisions that produce a suboptimal outcome for one group or service in order to reduce the total impact caused by the larger incident.

Alternatives

IRTs aren't the only answer to managing large and complex incidents. Some organizations adopt a consensus-driven model, and others ask executives and other leaders to run the response. Google's experience is that these two approaches lead to slower mitigation and more confusion and stress. As such, Google prefers using incident response teams, but the alternatives may be appropriate for other organizations.

Consensus-driven response

When incidents span teams, one option for coordinating the overall response is to bring all the subject matter experts together in a forum—a situation room, chat space, virtual meeting, or call—and let the group build consensus on mitigation options. This approach can work if the group is small, aligned on objectives, and has a culture of high trust and psychological safety, aligning with Westrum's model of a "generative culture" (see Chapter 4).

The major downside of this approach is that it may take a while for the responders to come to consensus. Additionally, it can lead to groupthink and bad outcomes if consensus is prioritized at the expense of psychological safety. Even worse, if the organization demonstrates Westrum's "pathological" culture, the consensus approach can turn political, with more assertive teams pushing the investigations in a direction that exculpates their team or blames others.

Executives as incident commanders

Some organizations default to having executives or other senior leaders act as de facto incident commanders rather than standing up a response team. The advantage of this approach is that executives are experienced at making difficult decisions and have more authority to garner resources. The challenge is that often executives have the least visibility of the technical complexity of the system and aren't well versed in mitigation op-

tions, including the preplanned procedures teams might have in place to deal with these exact scenarios. This leads to the executives pulling information from the responders, trying to form a mental model of the system in the moment, and disrupting established workflows and procedures, which slows the response. Additionally, it makes a small set of senior leaders bottlenecks—or worse, single points of failure—during outages.

Often, people want the executives to make incident response decisions out of fear that the engineers are missing business context. A better strategy is for the organization to make it possible for skilled responders to understand the business context through discovery, training, or frequent context sharing. As an example, Google occasionally has to make the decision to shut down certain racks in a datacenter to perform maintenance or reduce load on the cooling system. Being able to discover whether customers are running critical workloads in those racks is essential to making a quality decision about which rack to drain. As such, the information is queryable by incident response teams.

Additionally, the power that executives have to garner resources can be granted to an incident response team. Google allows its IRT members to redirect capacity, commandeer resources, and pull in anyone necessary through the inversion of authority model discussed earlier.

The Lifecycle of an Incident

Incidents can be stressful, high-stakes situations. To make incident response go as smoothly as possible, follow a structured process that covers the various stages of response: detection, triage, investigation, mitigation, resolution, and learning. It's important to note that incidents don't always proceed in an orderly fashion through these steps. Sometimes an attempted mitigation triggers new symptoms that are then detected, or the mitigation fails and requires more investigation. In really complex incidents, there may be multiple workstreams of investigations and mitigation attempts.

This section walks through the stages of the incident lifecycle, outlining the key activities and objectives for each phase. By breaking down the response into discrete steps, teams can focus on the right priorities at the right time.

Detection

Detection is the moment when some symptom of the incident is apparent to either a person or a software system capable of responding. Examples include:

- Automated alerts firing based on SLOs or other monitoring
- Customers opening support cases
- Engineers noticing the system is no longer behaving as expected

Ideally, most incidents are automatically detected. However, this is challenging in practice due to the sheer complexity of how users and systems interact. Chapters 8 and 9 offer ideas for addressing these challenges.

Triage

Triage is about understanding the severity of the situation and who needs to address it. The guiding principle here centers on the user experience. It can be tempting to jump to easy-to-measure signals like errors and latency as a means of resolving the early ambiguity of an incident. However, getting data that can tell you whether the people using the service are experiencing broken or severely degraded functionality is a higher priority in the triage process.

You can't triage situations you don't know about, so encourage *anyone* in your organization to declare an incident early. It's much better to overescalate and later recategorize the severity or stand down completely than to let an outage worsen because someone was afraid of being wrong or causing trouble. There should be no penalty (formal, cultural, or otherwise) for declaring an incident that turns out to be a false positive. The cost of investigating a declared outage is not zero, however, so the team responsible for triage should use automation, well-curated dashboards, etc. to lower this cost as much as possible. Watch out for political incentives too; if teams are punished for having outages, they will find ways to classify them as something else, which only hides the problem.

When responding to an outage, rarely do you have the luxury of all the facts. Effective triage under pressure means making quality decisions with incomplete information, not waiting for a perfect diagnosis. The process starts by asking a few critical questions:

- How many users are affected?
- Is this a global issue or isolated to a single location?
- What critical user journeys are broken?

The answers to these questions help decide the level of urgency and the tolerance for risk-taking during mitigation attempts. An incident that is widely visible to users can often be approached with a higher risk tolerance when choosing a mitigation strategy. An

incident that is causing only minimal impact to users should be treated with more caution because the opportunity to accidentally make things worse is higher.

During triage, accessible, clear outage data is more valuable than comprehensive information hidden in logs. Additionally, the following practices improve triage speed and accuracy:

- Avoid overindexing on a single source of information.
- Integrate automated alerts with customer support reports for a clearer view of impact. See also "The Role of AI in Incident Response" and "Case Study: Detectr".
- Use consolidated dashboards to view key metrics together rather than switching between tools.

Tools and dashboards should be prepared in advance, and the team must be proficient with them.

Success in triage is not about finding the fix; it is about speed to accurate classification and appropriate escalation. If you correctly identify the severity, mobilize the right responders, and stop the bleeding (or escalate to make this happen), then you have succeeded. Even if it turns out to be a false alarm, a rapid triage that leads to a quick standdown is a win, not a waste of time.

Investigation

The investigation phase of incident response covers the activities meant to help the response team form a mental model of the incident so that they can be more effective at identifying potential mitigation strategies. The priority here is finding a path to mitigation, not deeply understanding what went wrong. Depending on the complexity of the system and the organizational maturity, investigations tend to follow one of three approaches:

- Root cause analysis
- OODA (observe, orient, decide, act)
- Multiple hypotheses

Root cause analysis

The idea behind root cause analysis is to find "the one thing" that went wrong in the system and fix it. In this approach to investigation, responders tend to spend more time troubleshooting and debugging and may try to reproduce the issue in another environ-

ment. The main criticisms of root cause analysis (which we'll explore in more detail in Chapter 12) are as follows:

- In any reasonably complex system, it's very likely that multiple things will have gone wrong. Trying to find a single "cause" is reductive and may delay mitigation.
- Debugging and troubleshooting require deep expertise and the luxury of time. High-stakes incidents move fast, and the experts might not be available.
- Designation of a "root cause" can be political. Teams may refuse to join the response because they're not responsible for that part of the system. Leaders may fixate on one error or defect because it fits their biases about what constitutes bad design or bad behavior.

OODA

The OODA model, first introduced by US Air Force Colonel John Boyd, addresses the downsides of root cause analysis by organizing the investigation efforts around understanding the situation and taking action, regularly recalibrating your mental model based on new information. Incident responders use alerts, dashboards, logs, and system diagrams to observe and orient themselves. They decide on a course of action, execute it, and learn from how the system responds.

Critics of the OODA approach point out that it still suggests a linear model of incident response, in which responders are working with one mental model and action at a time. Additionally, the *orient* and *decide* parts of the approach are complex cognitive activities, and the model does not provide guidance on how best to approach them. Finally, in highly complex systems, it can be difficult to understand whether an action actually led to perceived change in system behavior—the observed effect might have been a coincidence.

Multiple hypotheses

The multiple hypotheses approach to investigations moves responders away from linear thinking toward regularly forming and testing new hypotheses to explain system behavior and identify potential mitigation options. In this approach, responders treat the investigation as a process of disproving hypotheses until they are left with one that can inform a reasonable and safe mitigation. The advantages are:

- Responders can pursue multiple ideas in parallel.
- The framing reduces confirmation bias by asking responders to disprove ideas rather than confirm them.

- Mitigation is often faster because the first mitigation to work wins.

Of course, it also has downsides:

- Multiple responders are needed to fully realize the efficiencies.
- There is a higher coordination cost to ensure that people aren't performing simultaneous mitigations that combine in unintended ways or create confusion.
- Coming up with hypotheses and experiments to test them is challenging work that requires a high degree of skills, experience, and expertise.

Despite the challenges, however, this is the preferred approach at Google, as we have found it yields faster mitigation and a more coordinated response.

Mitigation

A primary goal of incident response is to reduce the impact of the outage on the user by taking action. This goal takes priority over in-depth diagnosis. A mitigation-first mindset recognizes that the most damaging phase of any outage is the period in which users are experiencing breakage. Because expediency is critical during this phase, responders are often working with a limited understanding of the outage. Reducing the impact on users as quickly as possible—even if you can't fully eliminate it—is preferable to attempting to fully understand why a service is broken before taking action.

Mitigation is most effective with a toolbox of safe, "generic" strategies such as rollbacks, capacity increases, reboots, and traffic shifts, any of which can be applied broadly without requiring a deep understanding of the incident. Rather than being considered a separate phase, mitigation should be interwoven with investigation; applying a safe mitigation can act as a diagnostic experiment that reduces user pain while simultaneously ruling out some hypotheses. Once a mitigation succeeds, responders have space for a more thorough investigation.

The ability to safely mitigate incidents is ultimately an architectural property of the system. A system supports broad mitigation patterns only if its design is robust to specific emergency actions, such as rolling restarts, backward-compatible rollbacks, or rapid capacity increases. Additionally, the organization's culture must be one where reasonable actions taken during mitigation are never the source of blame or punishment, even if the actions inadvertently exacerbate the incident. Leaders must avoid outcome bias, or "resulting," where they judge actions not by the quality of the decision based upon the information available at the time, but by the results that could only be known in hindsight. In complex systems, cause and effect are not always obvious, and actions that appear

safe one day can easily be unsafe the next due to software bugs or changes in system assumptions.

During mitigation, safety is critical. Transparency, risk analysis, and predictable failure modes are required to reduce the likelihood of a mitigation making things worse. However, it is important to note that safety is not guaranteed through policy and compliance. More often, safety is achieved by having an organizational culture where individuals feel an ethical and professional responsibility and where the people closest to the information are empowered to make decisions. Additionally, all actions must be weighed against the risk of doing nothing; in outages, often the system is so broken that it's worth taking relatively safe but speculative actions to get it working again quickly.

Resolution

Once an incident is mitigated, the emergency phase ends and focus shifts to resolution. In this phase, the system is returned to the stable state it was in before the incident: rollbacks are fully stabilized, temporary mitigations are complete, and so on. Unlike mitigation, which typically involves applying a temporary fix or workaround without necessarily knowing all the factors that contributed to the incident, during resolution the responders are expected to have a deeper understanding of the outage, and the produced solutions should reflect that understanding.

In the resolution phase, the velocity of the response can—and should—slow down. The intense pressure to restore the service is lifted, and the remaining responders may take a more measured approach to finding a stable solution. This transition allows the incident commander to reduce the size of the response team, freeing up people to return to their normal duties.

For an incident to move from "Mitigated" to "Resolved" in Google's incident management tracking system, one or more of these conditions must be met:

- The primary cause or causes of the incident have been identified, and people are confident they won't recur.
- A stable fix has been implemented, or the primary breaking change has been rolled back and blocked from redeployment.
- The incident no longer requires an active, organized incident response because the system is in a good enough state that work can be done according to normal timelines and processes.

In complex incidents, it is often a best practice to declare a "wait and see" intermediate state after mitigation but before full resolution. This allows for a period of observation to confirm that the system is stable or for external parties or automated tests to verify that the impact has ended. In cases that involve a hierarchy of incidents, the parent incident might remain in this intermediate state until all child incidents are resolved. This prevents premature closure of incidents that might flare up again.

Once the incident commander, in consultation with other members of the response team, determines that the incident is fully resolved, they take the following actions:

- Conclude the emergency response measures.
- Explicitly communicate that the incident is resolved and closed on all channels used during the incident.

The focus then shifts to the final phase: learning from the incident through the post-mortem process.

Learning

After the incident is resolved, the responders pivot to documenting the outage in preparation for post-incident analysis, completing any remaining work, such as creating bug reports to track known issues and action items. This is also where planning begins for larger efforts to address the underlying failure mode to prevent it from happening again. (How organizations learn from incidents is covered in more detail in Chapter 11.) However, it's important to explicitly stress here that learning from the incident is a critical part of the overall response.

Additionally, capturing the responders' experience is essential to that learning process. As such, it helps if incident response tools and processes make it easier to analyze pain points during the response. Recording the names and roles of participants, capturing the alerts and signals that fired, preserving notes and chat logs, and automatically recording key timeline events will go a long way toward making post-incident analysis faster and more effective. This data provides a wealth of information about which signals turned out to be red herrings, which tools didn't work as expected, and which people ultimately needed to be involved to resolve the incident.

Collaboration and Communication

The ideal incident response is one where the alert goes straight to the person (or, as we'll see soon, agent) best equipped to analyze and mitigate the situation. However, we often don't get to experience this ideal, and many incidents require several people working together to resolve them. Unfortunately, as more people become engaged in a response, more effort is needed to communicate, coordinate, and ensure everyone is working from the same mental model. A response requiring tens—or worse, hundreds—of people might be a sign that the organization needs to invest in observability, training, or better practices. The goal is to have as many people as you need, but for that number to be as small as possible.

That being said, a good cultural norm during incident response is to encourage bringing in more people if responders feel stuck or need more expertise. In some organizations, incident responders might be afraid to page in more people, or they might not even know who to page. To minimize this friction, having a directory of all service rotations with clear instructions on how to reach out to the on-callers is helpful. However, in most organizations, there is still some amount of tacit knowledge required to identify the right people. This is another place where an IRT can be helpful: if nothing else, they know who to get involved.

On-callers should be trained that when paged, they are expected to be focused on the incident and remain engaged for as long as they can be useful. However, once they are no longer useful, they should be encouraged to disengage. Humans are notoriously bad at passive monitoring for more than 30 minutes at a time, which includes waiting for their names to be mentioned in chats and calls. A responder who hasn't been asked a question, provided input, or assigned a specific task in over an hour should be assumed to be disengaged.

Additionally, response tends to be smoother when people explicitly page in other responders, as opposed to using casual communication mechanisms like email or chat. Paging sends a clear message that you're urgently needed, and people shouldn't feel like they have to constantly check email and chats outside of business hours. Also, engineers need focus time to do engineering work; a page-first norm allows them to enter a state of flow knowing they will be paged if needed.

Encouraging responders to wait to be paged and to disengage if they are no longer needed can be counterintuitive. At Google, we favor a small number of incident response participants with clear roles and responsibilities. Folks are allowed to leave the response

knowing that if they are needed again, they'll be paged and asked to rejoin the response. Large organizations can make the mistake of engaging in "incident theater," where they treat the volume of people on the incident bridge or sitting in the situation room as a signal of good incident management when it is not. This comes at a significant coordination cost.

MEASURING THE EFFECTIVENESS OF INCIDENT RESPONSE

Organizations often want to measure whether their incident response is effective or not. Typically, they focus on metrics like mean time to resolution (MTTR) or mean time to mitigation (MTTM). The challenge with these metrics is not just that averages are misleading and fail to capture the variability of the data; it's that different incidents have wildly different expected timelines for response. An incident that involves a single service returning errors on 100% of traffic after a rollout should take minutes, or at most hours, to resolve. However, resolving an incident requiring a physical intervention of some kind—hardware repair, water evacuation, full power recovery—would likely take multiple hours or days.

Courtney Nash, who researches publicly available incident reports for the VOID, goes so far as to propose that measuring and *minimizing* the number of people or unique teams involved is a better metric for the effectiveness of incident response than common metrics like MTTR. This can seem counterintuitive because many assume that taking an "all-hands-on-deck" approach demonstrates the right organizational culture. However, the "more is better" approach to incident response ignores the opportunity cost of having too many engineers involved and increases the coordination costs.

In fact, Dr. Laura Mcguire talks about the "cost of coordination" as a way to reason about response effectiveness. While there is no single metric that captures this cost, understanding aspects like how long it takes new responders to come up to speed, how often responders are interrupted to provide status updates, and the number of different communication channels or tools involved in the response can lead to deeper insights on how to improve response.

Looking at the number of people involved, costs of coordination, or other activity-related metrics is a good way to find patterns that can lead to improvements in processes and tools. These patterns can provide deeper insights than high-level metrics such as MTTR or MTTM.

There are several antipatterns to watch out for here. In large organizations, stakeholders may join incident responses to try to expedite mitigation or to satisfy their curiosity. These well-meaning individuals can inadvertently delay mitigation by interrupting the flow of the discussion, distracting responders from key tasks, or, in the worst case, demoralizing them.

If people join the response and start engaging in the following behaviors, the incident responders should gently redirect them:

Repeatedly asking for updates, ETAs, or customer-specific information

> **Mitigation:** Responders should point them to where regular updates are being provided as part of the standard incident response process so they can follow along. This behavior is also a good sign that the response needs a communications lead (see the sidebar "Incident Management at Google").

Making demoralizing statements about the length or pace of the response

> **Mitigation:** Ask them to record their thoughts somewhere else and share them as constructive suggestions during post-incident analysis.

Criticizing the system design or proposing major changes—e.g., rearchitectures or new features

> **Mitigation:** As in the previous case, redirect them to contributing their ideas to the post-incident analysis.

Asking frequent, rudimentary questions about the system

> **Mitigation:** Point them to other resources for coming up to speed (e.g., search, documentation, or AI tools), or pair them with a more experienced responder.

Interfering with incident command by giving conflicting directions or overruling decisions

> **Mitigation:** Give them a specific task to focus on, or, if they are better suited to lead the response, offer to hand over the incident commander role to reduce confusion and speed the response.

Training and Development

Incident management requires a set of skills that improve with experience. Fortunately, those skills can also be developed or improved *outside* of incidents.

Techniques for incident management and response were covered extensively in Chapter 14 of the first edition of this book and Chapter 9 of *The Site Reliability Workbook*. Here's a brief recap:

Disaster Resilience Testing (DiRT)

These periodic, organization-wide resilience exercises (formerly referred to as Disaster Recovery Testing at Google) validate system recovery and practice large-scale coordination. Integrate your incident management procedures into these tests to evaluate how teams handle simulated emergencies under realistic, cross-functional conditions. For more information, see "Disaster Recovery and Resilience".

Wheel of Misfortune

To build muscle memory and familiarity with diagnostic tools in a low-stress environment, incident response teams can regularly participate in localized role-playing simulations (see Chapter 5).

Production reviews

These regular meetings analyze recent outages, review paging events, and track postmortem action items as a team (see the sidebar "Production Meetings"). Analyzing operational history in a collaborative setting accelerates the development of systems thinking and helps engineers calibrate their technical judgment.

Onboarding and ongoing education

The skills described in this chapter can all be taught, making structured onboarding crucial to the success of new SREs on your team. Seasoned responders also benefit from periodic refreshers as systems and tools inevitably drift over time. Google has developed SRE EDU (see "Formal training programs"), a peer-driven education program created and run by practicing SREs, for exactly this purpose.

The Ghost in the Machine

Managing the human element of an outage is just as important as (if not more important than) managing the technical aspects of an outage. Experience has taught us that the following less-obvious skills contribute to successful incident resolution:

Quick judgment

Making fast, quality decisions with limited information

Remaining calm in a crisis

: Staying composed in high pressure situations

Command-style leadership

: Explicitly delegating work and aligning people on a common mental model

Systems thinking

: Keeping a holistic perspective and understanding feedback loops

These skills are distinct from the education, training, and experience most engineers receive in their day-to-day work. This section examines what it takes to deliberately develop them for incident response.

Developing judgment

Decision making in an incident response requires thinking critically, evaluating evidence, checking for bias, and weighing trade-offs. However, these steps must happen quickly and with limited information. To improve the speed and quality of decision making in incident response, responders need to calibrate their judgment.

One of the best ways to improve judgment is to learn how others make decisions in similar situations. One strategy is to have an experienced and knowledgeable responder with a track record of quality decision making "think out loud" when faced with a tough decision. An observer listens to how this "high-judgment individual" processes information, weighs alternatives, and makes choices quickly. This allows the observer to calibrate their own decision-making processes. Shadowing is one way to create opportunities for this type of learning, but it works only if the experienced incident responders are deliberately explaining their thought processes.

Scenarios and simulations, as discussed in Chapter 5, also create opportunities to practice and calibrate judgment, as long as the participants are reflecting on their thought processes and getting feedback.

Staying calm in crisis

Incident response depends on individuals being able to work together effectively under stress. To avoid the response getting derailed by frustration or fear, the responders need to stay calm regardless of whether the incident involves a single service, a multiservice cascading failure, or an actual disaster.

While some people are naturally good at being calm in crisis, for those who aren't, a commonly recommended tool is practicing *mindfulness*. Mindfulness involves a variety of techniques, but generally, the practice teaches people to observe, name, and release negative thoughts or feelings so they can focus on being present in the moment. Some of the most experienced incident managers at Google have found that regular meditation strengthens the mental muscle needed to pause, reset, and focus when frustration or fear is getting in the way of incident response.

In addition to mindfulness, using the incident response tooling and processes *outside* of stressful situations helps build calm during an incident by removing an additional source of stress: if everyone in the response is well versed in the tools and processes, no one is struggling to learn them during the outage. (This is also an argument for investing in the usability of incident response tooling.)

Finally, the incident lead can help here by making it safe for people to step away and take a small break if needed, even going so far as to kindly recommend that they do.

Adapting leadership skills

While incident response requires standard leadership skills and practices such as communication, delegation, and transparency, they must be adapted for the high-pressure environment of an incident. In typical leadership scenarios, such as being a technical lead or manager, one might adjust their leadership approach to suit an individual's style. However, effective incident response demands that leadership take a consistent "command and control" approach, being clear and explicit while still maintaining psychological safety.

Delegating work to the responders is the first way this difference shows up. In a routine team meeting, a leader might seek interested volunteers by asking, "Who wants to investigate this issue?" Conversely, an incident commander should assign tasks directly to specific individuals with clear expectations to avoid the diffusion of responsibility. They might say, "[Name], please investigate the latest service rollout and report if its progress matches the timing and locations of our increased latency." In larger incidents, the incident commander should refrain from taking on tasks themselves in order to focus on coordinating the response. This may feel awkward for leaders who like to take on unrewarding or toilsome work or deferring to their teams as a way of demonstrating "servant leadership," a leadership approach characterized by managers sharing power and focusing on team needs first.

Furthermore, the incident commander must ensure all participants share the same mental model and facts. This requires a succinct, direct communication style, frequently repeating critical information and reevaluating assumptions as new information arrives: "Just so we're all clear, this evidence means we can rule out the rollout as a contributing factor."

A great way to get everyone aligned on a mental model and to make sure people stay focused on the goals is to ask pointed questions. For instance, asking "Is there anything we can do to speed the rollback?" focuses the response on an objective without getting into the details of how the rollback system works. This can also be a way to make it safe for people to ask for assistance. For example, you could ask, "Would it help if we have someone join you to help you troubleshoot?"

Developing this explicit leadership style requires observing effective incident commanders, practicing outside of the stress of a real incident using simulations or role-playing, and potentially, getting coached by seasoned responders. Simulations and reverse shadowing serve as excellent methods for gaining this experience and receiving constructive feedback.

Cultivating systems thinking

Incident responders have to constantly zoom in and out on the system as they evaluate hypotheses and incorporate new information. They must be able to recognize feedback loops and reason about how the hardware, software, people, and processes all interact. This skill is commonly referred to as "systems thinking." You can grow your ability to "think in systems" by practicing modeling complex systems, identifying feedback loops, and reasoning about system behavior.

Teaching this is much like teaching judgment—people learn from skilled and experienced practitioners as they "think out loud." Having responders follow along in analyses like CAST or STPA, covered in Chapter 12, is another effective way to teach systems thinking.

The Role of AI in Incident Response

As AI's role in systems development grows, to keep pace with the velocity and inevitable complexity, incident response needs to incorporate it as well. This section describes some areas benefiting from the early application of AI.

The introduction of this technology is understandably being met with excitement and a degree of trepidation as the models are getting increasingly good at performing engineering activities—and as with any new technology, its accuracy and safety are being heavily scrutinized, particularly within SRE. Our focus here is specifically on incident management, but we discuss the integration of AI into SRE more broadly in Chapter 21.

Introducing AI—and particularly, autonomous agents—into the domain of incident response can be a gradual process that minimizes risk.

Detection and Triage

Triage and detection are relatively safe places to introduce AI because with the right framing (assume the worst until proven otherwise), the biggest risk is that an incident is overescalated. With additional guardrails, covered in Chapter 21, even the risk of overescalation can be reduced.

Traditional incident detection relies heavily on standard telemetry such as metrics, logs, and traces, which are good at catching known failure modes but can miss novel issues or those that primarily manifest as user pain without triggering static alarms. As discussed in "Case Study: Detectr", AI and large language models (LLMs) can serve as a powerful semantic addition to traditional telemetry. By surfacing insights from unstructured customer feedback, AI tools help responders quickly identify novel or emerging failure patterns that purely statistical anomaly detection might miss.

Outage Communication

Even at this early stage, another effective place to introduce LLMs is in producing summaries of outages for stakeholders and responders who are coming up to speed on the current state of the incident. Rather than interrupting the response flow with questions, interested parties can ask for AI-generated summaries of transcripts, chats, and working documents.

Investigation

AI can generate dynamic investigation dashboards on the fly, customized for an ongoing incident. By analyzing historical similarities, the context of the triggered playbook, and the logical intent of the alert, the agent dynamically assembles a single pane of glass containing the most relevant graphs and logs for that specific outage.

Hypothesis Generation

To accelerate the investigation phase, AI can automatically generate a high-confidence incident hypothesis. By ingesting real-time monitoring anomalies, application logs, and patterns from similar past incidents, the agent provides the on-caller with a single credible lead and concrete verification steps. Alternatively, AI agents can propose and test hypotheses in a multiple hypotheses approach. They can also serve as "rubber duck debuggers," helping responders check their assumptions about how the system actually works against their mental model of how it should work.

On-call Handoff

The communication overhead during an incident scales with the number of responders involved. Agents can monitor communication surfaces such as chat spaces, video transcripts, and incident documents to passively synthesize the state of all activity. They can summarize exactly what was tried, what failed, and what tools were used, and importantly keep the most relevant data in the forefront as the picture of the incident comes into focus. This allows new responders to get caught up rapidly without distracting the active incident commander.

Autonomy in Production

Before allowing AI to participate in mitigation efforts by making changes in production, it is important to have sufficient guardrails and observability in place and to ensure that you're following robust change management practices. Any agents performing AI operations should have robust evaluation mechanisms and feedback loops that ensure agents perform as expected and produce quality results. Other considerations include identity and authorization for agents.

Any change made to production by an autonomous agent during an incident should be reversible. Each change should be treated as an atomic operation: the action, immediate validation, and, if validation fails, an automatic rollback should be executed as a single, auditable transaction. To ensure transparency and provide input to further improvement, agents should log their complete reasoning, including signals used and their confidence levels.

It is important to note that processes and operations that are already automated or can easily be automated with classic non-AI-based systems do not need to be replaced with AI versions as long as they meet business needs. Traditional automation is easier to reason about and repair if it starts misbehaving.

Additionally, since agents are a form of automation, all of the considerations covered in Chapter 13 apply to agents as well. Most notably, for the use of AI to be safe and effective, the underlying production systems must be designed for machine interaction.

A North Star for Agent Safety

Looking to the future, the goal for agentic incident response is to have agents handle routine or low-complexity incidents with little to no human intervention, reserving the human responders for more complex and novel situations where their involvement is needed to resolve ambiguity and create safety. As AI improves, more and more incidents should be handled by agents. This is a natural progression of SRE's philosophy of scale through automation.

A major consideration in adopting AI in incident response is how much human oversight is required. A range of approaches and opinions have been published on the necessity of a human in the loop, particularly questioning whether the human should act as an *approver* of AI actions or an *observer*. Some proposals for AI incident response suggest more of an observer model, skipping human approval for routine tasks under the assumption that as systems get more and more complex, the approval process will bottleneck response. Others suggest humans must approve every agent action, overlooking the fact that approval processes, particularly those that come late in the process, can become a form of "safety clutter" and lead to "approval-by-default" behavior because understanding what is being approved requires significant cognitive effort.

Rather than adhering to one extreme or the other, organizations should adopt a balanced and progressive approach. Initially, agents can be allowed production-safe, read-only access; then, potentially, authorization to make safe, routine changes. More complex changes should initially require human review until the agents demonstrate sufficient success. This human-in-the-loop "review and approve" model is not toil but rather feedback to safely train the model for higher levels of autonomy, culminating in end-to-end performance of certain tasks in production with full autonomy. This approach nicely aligns with SRE principles of production safety and automation.

As AI-assisted tools allow developers to deliver dramatically more code at faster speeds, traditional production automation will no longer be sufficient to scale these systems reliably. SRE will certainly adopt these same powerful tools, and incident management and on-call are excellent early candidates for adoption. The opportunity is immense! It is an exciting time to be an SRE.

Case Study: Incident Response and Management at Etsy

By Nick Bitzer

Etsy is the global marketplace for original and creative goods that connects a community of makers, artisans, and creative entrepreneurs to millions of buyers looking for something special. Behind our platform sits a complex engineering organization that processes billions of events each day—from clicks and favorites to adds-to-cart and purchases—while maintaining the reliability expectations of a modern ecommerce business. What sets Etsy apart is not just the scale of our technical operations but the cultural foundation that enables our engineering teams to respond to incidents effectively and collaboratively.

This case study examines how we have evolved from quietly handling incidents in team-specific channels with low visibility to embracing radical transparency (when possible) across the entire organization. This transformation demonstrates that effective incident response is built primarily on cultural investments—namely psychological safety and blameless practices—before technical tooling or process improvements can take hold.

Our approach offers valuable lessons for organizations of any size, demonstrating the power of the belief that everyone is doing their best when they show up to work, that incidents provide opportunities for improvement rather than punishment, and that transparency during high-stress moments produces better outcomes.

The Cultural Foundation

At Etsy, our incident response program relies on the principle of psychological safety that permeates the entire engineering organization. This is not merely a policy statement; it is a lived practice (evident throughout all contributor levels) that determines how our engineers and leadership behave during the most stressful moments of their work.

Blameless culture as bedrock

Our commitment to blameless culture enables the transparent incident process that has become a cornerstone of our engineering organization. The philosophy is straightforward: even though everyone is doing their best, incidents will happen, and that is acceptable. This perspective differs markedly from that of blame-focused organizations in

which leaders pressure teams for quick resolutions, often at the expense of allowing experts time to identify the causes of incidents without fear of retribution.

Engineers who fear shouldering the blame for an incident are naturally disincentivized to publicly deconstruct the failure mechanism with transparency. But for an engineer whose actions contributed to an incident, a blameless culture leaves no room for shaming. Rather, they are encouraged to provide detailed accounts of what actions they took, what effects they observed, what expectations and assumptions they held when making a change, and their understanding of the timeline, all without fear of punishment.

Etsy's trust culture and psychological safety

Our engineering culture operates with extraordinarily high levels of trust. We do not enforce a "get approval first" culture but rather favor autonomy for engineers at all levels to ideate and drive solutions they deem valuable. This trust extends directly into incident response; our goal is that no one fears opening an incident, regardless of its severity or their personal level of experience. From junior engineers to VPs, when something happens, everyone at Etsy is encouraged to open an incident.

Psychological safety is reinforced through visible executive support. At Etsy, it's common to find all levels of engineering leadership participating in incident channels, modeling the right behaviors and demonstrating why incident response is everyone's responsibility, not a specialized function delegated to certain teams or levels. When leadership shows up during incidents, not to assign blame but to contribute or provide support and direction, it signals to the entire organization that transparency is a core value in the incident resolution process.

Our Incident Response Evolution

Many years ago, our incident process looked quite different from how it does today. Incidents—especially those of lower priority—were generally handled quietly in team channels, with limited discussion in broader channels unless they were truly critical (e.g., a full-site outage). We were using homegrown tooling to manage the incident process, which had limitations (our core competency, after all, is building a marketplace, not incident management software). There was limited visibility into noncritical issues; incident information was difficult to obtain, and metrics were not widely reported across engineering.

We did, however, always maintain a strong postmortem process, although the information around incident metrics remained opaque to most of the organization. Teams had a

shared understanding of incident severities and customer impact, but the broader patterns were not yet being identified.

Over the last few years, we have accelerated our purposeful shift toward radical transparency in incident management processes when possible. Critical incident metrics (e.g., total number of incidents, total number of follow-ups from incidents, number of postmortems held) are now reported monthly to senior leadership and, barring any sensitivities, made available via a company-wide slide deck for everyone at Etsy to view. All non-sensitive incidents, no matter their severity level, are announced in a central Slack channel (#incident-announce) with roughly 1,000 members, including nearly the entire engineering organization and senior leadership. High-severity incidents (SEV1 and SEV2) are announced to even broader forums with more than 1,200 members.

Our cultural messaging has evolved alongside these process changes. We now promote these key principles:

- Everyone sees incidents; this is normal and desired.
- Everyone is empowered to open an incident.
- If you notice even the slightest indication of an incident, just open one and include anyone you suspect might be able to help.
- No one will ever get in trouble for opening an incident.
- Do not let an incident go to waste. Learn from it.

This transformation required dedicated time and bandwidth from senior leadership during the initial rollout. The cultural investment over time has paid off: our blameless and supportive cultural foundation made the technical evolution possible, not the other way around.

Incident Response in Action

With a supportive cultural foundation established, the underlying principles of incident management become practical steps during an actual outage. The following sections outline how this organized response operates in practice, beginning the moment a problem is first noticed.

Detection and declaration

We implemented an extremely low barrier to opening incidents. Simple tooling integrated into Slack (where everyone already communicates) allows engineers to quickly declare incidents and automatically create a shared coordination space. We also publish

guidance around severity levels to bring clarity to engineers who are opening incidents, but the cultural norm remains: *When in doubt, open an incident*. We've set up our tooling to ensure safe defaults when declaring an incident, allowing engineers to open incidents and adjust relevant information such as severity levels and impacted services later. Each incident receives a dedicated Slack channel upon declaration, clearly establishing where individuals should go to support incident resolution, gain context, and follow along. With this system, we are able to centralize incident context in a single place, allowing for more efficient communication and follow-up.

Our tooling supports not only manual creation but also automated detection of incidents. Automated systems can create incidents from monitoring alerts, while manual creation handles those "something seems wrong, but I don't see any alerts" moments. This dual approach acknowledges that complex systems fail in complex ways that monitoring cannot always anticipate.

With the recent evolution of agentic tooling, we have begun exploring how AI might help us make sense of large amounts of context and data during incidents. These early explorations position AI as an additional tool in the incident process. While we are still evaluating the value of these approaches, we see potential for AI to provide an extra level of support to engineering teams as they work to identify incident causes and resolutions.

The LSE coordinator role

We created the large-scale event (LSE) coordinator position, a role staffed from a small group of senior engineering leaders who possess both the business and technical context to make difficult decisions during incidents. This role deliberately separates business decisions from technical problem solving, allowing engineers working on the incident to focus on service restoration while coordinators handle stakeholder communication and business impact assessment.

The LSE coordinator position is staffed voluntarily, with members participating in an on-call rotation to ensure availability. The staff are distributed across our global teams, with a single coordinator on a 24×7 rotation for a week at a time. We publish documentation that clearly explains the role and how to engage the on-call coordinator, reducing friction when help is needed. For high-severity incidents, our tooling automatically pages and adds the LSE on-call, removing the burden of decision making from engineers already managing an incident.

Cross-organizational collaboration

We promote a “pitch-in” culture during incidents. If you believe you can add value, you are encouraged to engage through established incident response processes. Engineers and leaders from multiple teams often collaborate to support incident resolution. During incident response, there is a strong sense of shared ownership. People step in to support where they can, approaching issues as collective responsibilities across Etsy.

To foster our pitch-in culture without sacrificing clarity, we assign an incident lead. This role is crucial for ensuring we avoid the issue of too many competing opinions muddying the situation. The incident lead is responsible for supplying regular updates in the incident channel, doling out tasks when needed, and keeping engineers focused on the core problem at hand.

Our Postmortem Process

Upon resolution of every incident, our tooling asks if a postmortem should be held or not. We have a rule of thumb that all SEV1 and SEV2 incidents must have a postmortem due to the level of impact, but for other severity levels (SEV3 and SEV4) it’s optional, and the decision is left up to those involved in the incident. This small reminder reinforces the importance of learning from incidents and reduces the friction of remembering that a postmortem may be needed.

Every postmortem opens with a standard statement reinforcing blameless culture. The focus is on learning, not punishment. Terms like “root cause” are avoided, as is naming specific people who might have contributed to the incident. The meeting centers on replaying incident events without finger-pointing, asking, “How do we get better?” rather than, “Who messed up?”

We run a postmortem facilitator training program, with all postmortems led by engineering individual contributors across the organization. This approach helps ensure the discussion focuses on systems and processes rather than hierarchy—the problem, not the people.

New facilitators are onboarded regularly, and postmortem facilitator efforts are celebrated throughout the year. Participation is self-driven; it is not a requirement for any engineer to be a facilitator. Engineers run the entire process, from scheduling and meeting organization through facilitation, wrap-up, and organizational communications.

Lessons for Other Organizations

We believe our experience offers some transferable insights:

Culture is paramount.

Our supportive, open culture is the foundation of our successful incident program, and without it, no amount of tooling would have produced the same results.

Transparency requires psychological safety.

You cannot ask people to be transparent about their mistakes if visibility leads to punishment.

Leadership presence and modeling matter enormously.

When members of senior leadership participate in incident channels and post-mortems, they signal that these activities are valued, not merely allowed.

Tooling and processes must be low-friction.

If the barrier to opening and managing incidents is high, people are less likely to utilize the system. The bar should be low enough to trip over.

Perhaps most importantly: small organizations can start here, and large organizations can arrive here. Cultural transformation is possible regardless of scale, though it requires patience, consistency, and genuine commitment from everyone involved.

Takeaways

Our incident response journey demonstrates that an outstanding incident process is achievable with the right cultural support. We continue to refine our approach, recognizing that incident management is an ongoing journey rather than a destination. For organizations seeking to improve their own incident response capabilities, our story offers a clear message: invest in culture, not just process. Build psychological safety before building dashboards, and create an environment where people feel safe being transparent about failure. The tools and processes will follow.

Conclusion

Incident management is a sociotechnical discipline blending the structure of preparation and a growing set of tools with human understanding and adaptation. This chapter ex-

plored fundamental practices for incident management and on-call, which are both crucial for maintaining reliable products. As you approach on-call and incident response within your own organization, keep the following takeaways in mind:

- Adopting or defining a structured incident response model that is regularly practiced outside the stressful environment of a live incident makes for a more effective response.
- Incident response requires developing skills beyond technical expertise, including the ability to make fast decisions, stay calm, and lead others without compromising psychological safety in high-stress, fast-moving situations.
- AI is a useful tool for responders and should be integrated strategically throughout the response lifecycle, while recognizing when humans are better equipped to lead the response.

During incident response, there is intense pressure to move fast and fix things—and if the right structures are not established, incident response can be chaotic and slow. Ultimately, reliable incident management is not just about technology but about cultivating a capable, calm, and empowered culture of responders. The old saying, "Slow is smooth, and smooth is fast" very much applies.

1 Paul Cichonski et al., "Computer Security Incident Handling Guide," NIST Special Publication 800-61 Revision 2, 2012.

Previous chapter
‹ 9. Observability and Monitoring

Next chapter
11. Learning from Incidents ›